TAKTILE

Trustworthy claims triage: keep decisions right as the rules change

Taktile case proposal · thinc! Munich hackathon · 7–8 November 2026

About Taktile

Taktile is the decision platform that banks, insurers and fintechs use to build, test and run automated decisions such as credit, fraud and claims workflows. Every rule change in production goes through versioning, testing against past cases and monitoring after release. This case is a small version of that daily work. The Taktile engineer mentoring you builds these systems with customers every week, and we are here to meet people who want to do the same.

The problem

An insurer receives claims with a form, repair quote, policy details and supporting evidence. Some are straightforward; others are incomplete, inconsistent or outside routine policy. Reviewers spend time finding the relevant facts and deciding what should happen next. Policy and legal requirements also change. A system that was accurate last month can quietly become wrong.

The insurer wants faster handling without losing control of decision quality. The challenge is to help people make sound, explainable decisions today and keep those decisions sound when the rules change.

Who it is for

A claims reviewer checks individual cases and corrects recommendations. A claims policy owner defines the rules and approves changes. An operations lead watches quality, workload and the effect of changes across many cases.

Your mission

Build a working prototype that reads a fictional claim and recommends one of three next steps: ready for routine processing, request more information, or send to a human reviewer. Show the facts and policy behind the recommendation. Make uncertainty visible; never present a guess as a confirmed fact.

At a fixed time announced at the case presentation (planned for Saturday 18:00), you will receive a fictional policy change. Update your system, find open decisions that need another look, leave closed decisions as they were, and show whether the new version fixes the intended cases without making other cases worse.

Four deliverables

1. Intake and decision. Accept a claim pack, extract the relevant facts, apply the current policy, and produce a recommendation with a short, inspectable explanation. The structured fields decide the outcome, so a rules approach is valid and expected to score well. A language model may extract facts, draft explanations or help reviewers, but it must never override the written policy. Blank fields are absent facts: do not invent them.
2. Human review and change control. Give a reviewer a clear way to inspect evidence, correct a recommendation and record why. Give the policy owner a way to update the rules and see which past or pending claims the change affects. Keep the policy version attached to each recommendation.
3. Quality measurement. Test on labelled examples before and after the change. Show where the system gets decisions wrong, especially cases it incorrectly sends to routine processing. Explain how the insurer would keep measuring quality after launch, when real outcomes and reviewer corrections arrive later.
4. Business case. The insurer is impressed by the proof of value, but before funding a rollout it wants to understand the business case behind it. Build a simple, transparent model (a spreadsheet or a page in your app is enough) using transparent assumptions, which are realistic for mid-sized European motor insurers and are available for review as well.
- the annual cost of triage today and with your system, the net benefit and the payback period
- the cost of errors, using the error rates you measured in building block 3 under v1 and v2. A wrong routine-processing recommendation is far more expensive than an unnecessary escalation, so show how this shapes where you set thresholds
- the cost of a policy change today and with your system, using the v2 change as the example

What to demonstrate on Sunday

Show the working flow on at least one easy case, one incomplete case and one case affected by the rule change. Then show a before-and-after quality report. Include a short explanation of the most important trade-off you made and the next test you would run before using the system in production.
A good prototype may be a small web app, a command-line workflow or a combination. A live production integration is not required. We care more about sound decisions, clear evidence and useful measurement than visual polish.
The batch path matters as much as the screen: see Practical details for the required input and output format.

How we will assess decision quality

Use a labelled test set and report, by policy version: the share of correct next-step recommendations; the rate of incorrect routine-processing recommendations; and the rate of unnecessary human escalations. Show results for the changed cases separately, so a strong overall score cannot hide a failure on the new policy. Some cases have blank input fields; report how your system handles them.
For ongoing use, propose a sampling and review process. Say what signal would trigger investigation, who would approve a policy change, and how you would rerun previous cases before release. Reviewer overrides are a key signal, but the supplied data contains none, so show how your system would record them rather than reporting a rate. If the true outcome arrives weeks later, show how you would incorporate it rather than treating the initial recommendation as ground truth.

What Taktile provides

The following materials will be provided for you with this introduction:

- Fictional policy v1
- Labelled development cases (14)
- Unlabelled practice cases (12)

Judging criteria

1. Decision quality (30%): Does the system make the right next-step recommendation on the hidden cases, including after the change and when input fields are blank?
2. Handling uncertainty and change (15%): Does it escalate appropriately, preserve evidence and policy versions, identify affected open cases, and leave closed cases untouched?
3. Reviewer experience (20%): Can a person understand and correct the recommendation quickly? Judges will perform one live override and check that the correction, its reason, the reviewer and the policy version are recorded.
4. Measurement and learning (10%): Are the tests meaningful, are failures visible, and is there a credible plan to maintain quality?
5. Business Case (25%): Is the business case realistic, convincing and well-presented?

If AI makes a decision process faster, how does an insurer prove that it remains correct, fair and controllable as its policies and obligations evolve? That is the problem your prototype should help answer.

Practical details

Tools and models.

- Use any language, framework or AI tool. Rules are a complete and valid solution; a language model may extract facts, draft explanations or assist reviewers, but it must never override the written policy.
- Bring your own API keys unless thinc! announces credits.
- Submission. Before the Sunday review, share a repository link and have your prototype ready to run on a new CSV of claims in the provided format. It must write a results CSV with the columns case_id, recommendation_v1, recommendation_v2, needs_recheck, rule_v1, rule_v2 and explanation. For closed claims, recommendation_v2 repeats the retained v1 decision and needs_recheck is False. Judges score every team on the same hidden cases with a script, so this batch path is required.

Questions

Ask in the event chat or find the Taktile mentor on site. We answer questions about the policy and the data; we do not confirm labels for the practice cases.

Every member of the winning team in this case gets…

1) …a fast-track interview at Taktile (you need the right to work in that country)
    a) Open roles are a 6-month Software Engineering internship in Berlin starting March or April 2027
    b) a 3-month internship in Iasi in summer 2027, or
    c) for graduating students, a full-time Junior Engineer role in Berlin, Iasi, London or Dublin
    d) for graduating students, a full-time Forward Deployed Engineer role in Berlin, Munich or London
2) ..an Arduino ESP32 incl. Breadboard and Cables for your next project (incl. 40 sensors once per team to choose from)
3) ..Taktile Hoodie